Programmation Orientée Objet en **C++**

Emanuel Aldea <emanuel.aldea@u-psud.fr>
http://hebergement.u-psud.fr/emi

M1-E3A

Troisième chapitre

Les arbres

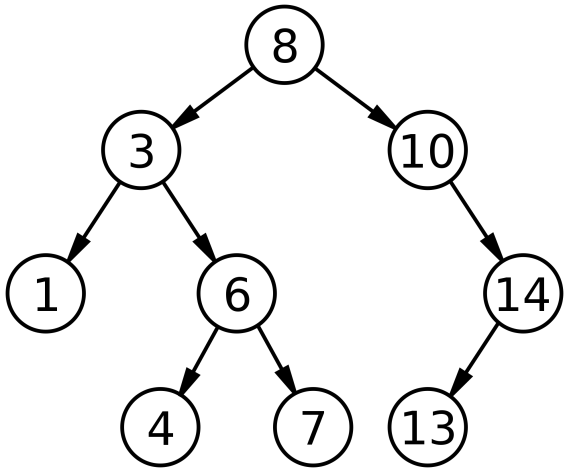
Justification

Des structures de données utilisées énormément en algorithmique :

Justification

Des structures de données utilisées énormément en algorithmique :

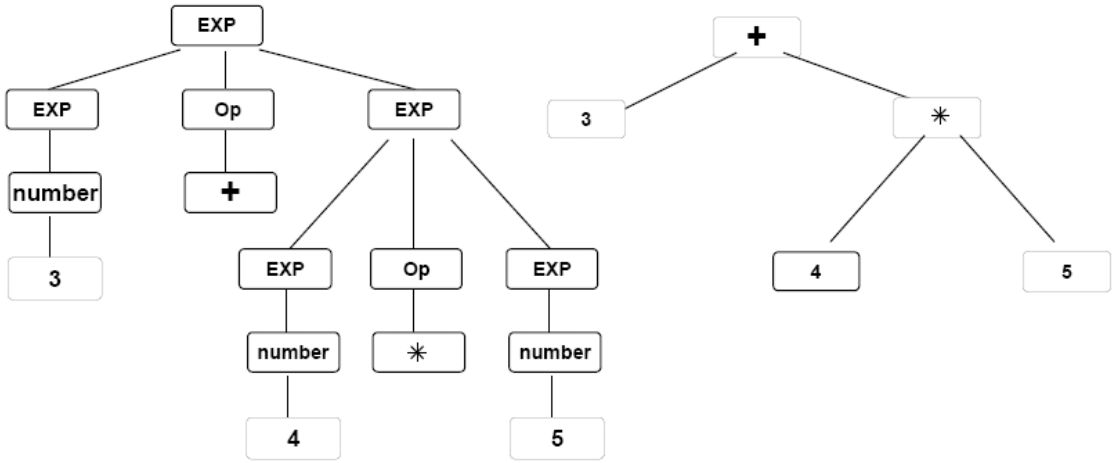
- les arbres de recherche, pour stocker et récupérer des données ordonnées



Justification

Des structures de données utilisées énormément en algorithmique :

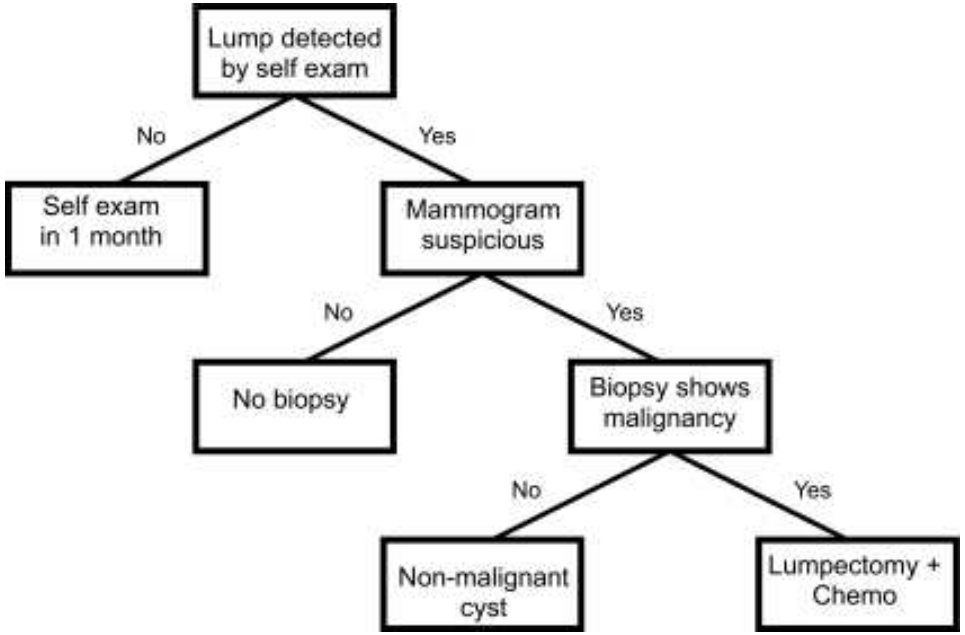
- les arbres de recherche, pour stocker et récupérer des données ordonnées
- les arbres syntactiques, utilisés par les compilateurs pour l'évaluation du code source



Justification

Des structures de données utilisées énormément en algorithmique :

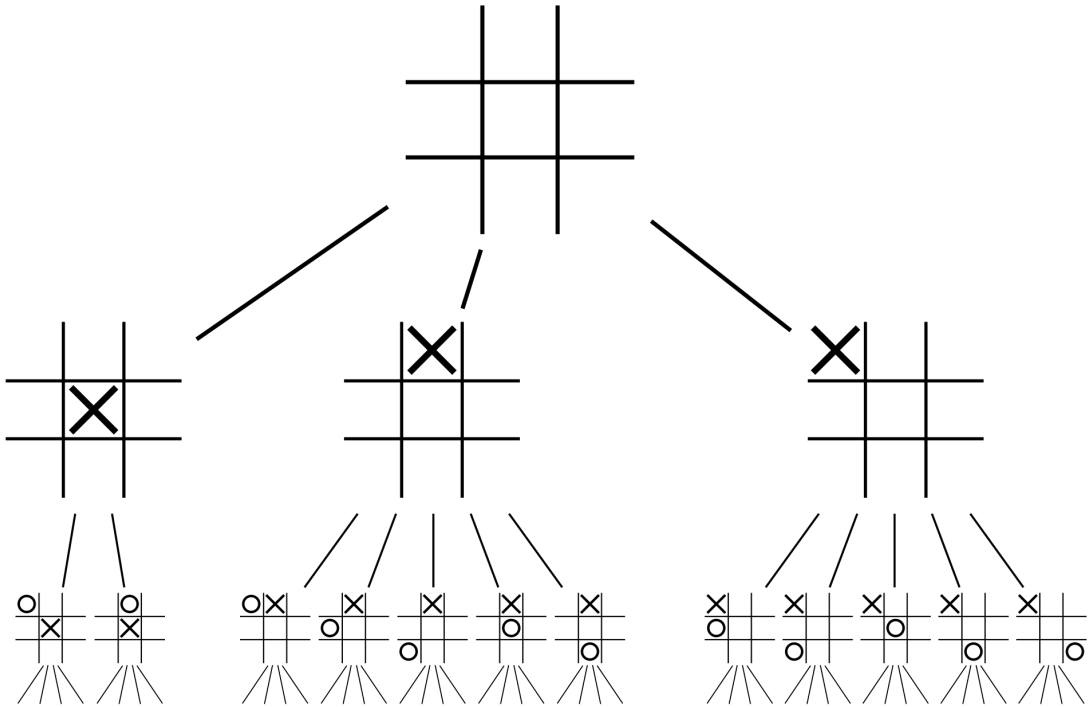
- les arbres de recherche, pour stocker et récupérer des données ordonnées
- les arbres syntactiques, utilisés par les compilateurs pour l'évaluation du code source
- les arbres de décision, utilisés en intelligence artificielle



Justification

Des structures de données utilisées énormément en algorithmique :

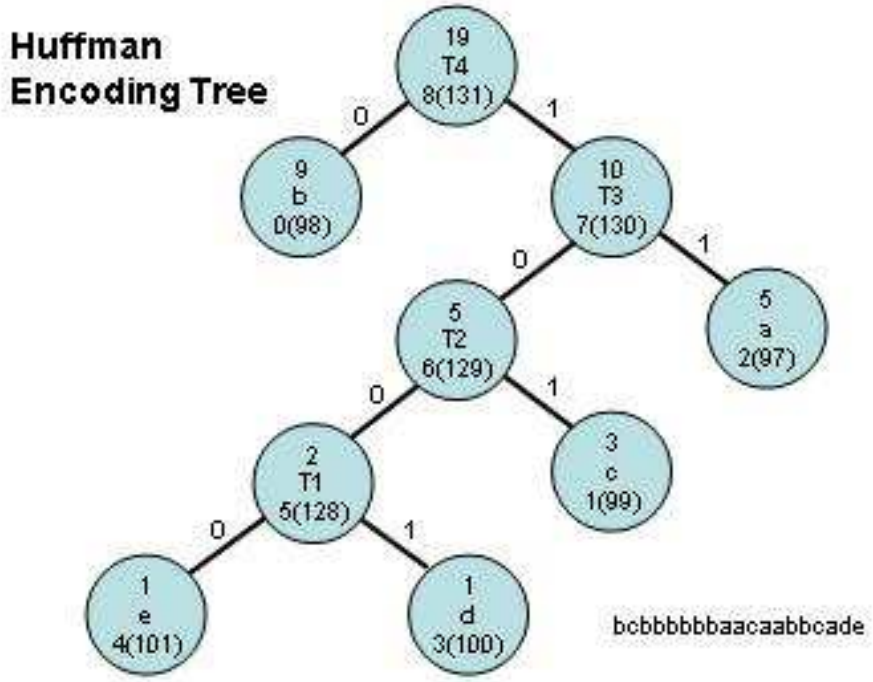
- les arbres de recherche, pour stocker et récupérer des données ordonnées
- les arbres syntactiques, utilisés par les compilateurs pour l'évaluation du code source
- les arbres de décision, utilisés en intelligence artificielle



Justification

Des structures de données utilisées énormément en algorithmique :

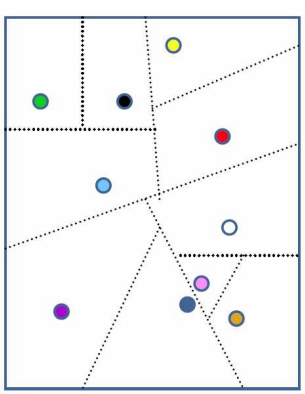
- les arbres de recherche, pour stocker et récupérer des données ordonnées
- les arbres syntactiques, utilisés par les compilateurs pour l'évaluation du code source
- les arbres de décision, utilisés en intelligence artificielle
- les arbres de codage, utilises en théorie de l'information (i.e. le code Huffman)

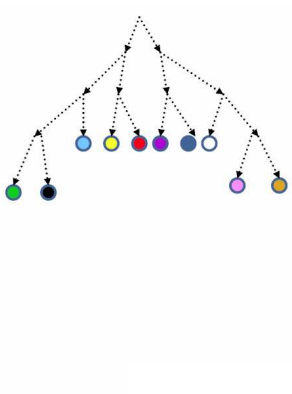


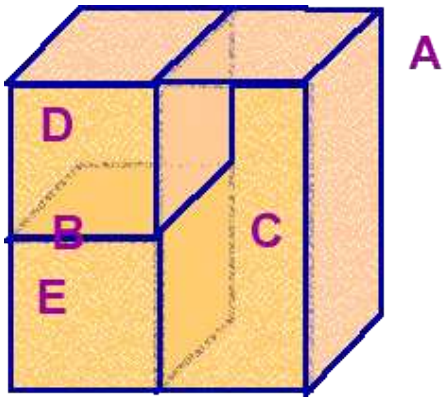
Justification

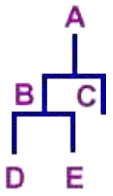
Des structures de données utilisées énormément en algorithmique :

- les arbres de recherche, pour stocker et récupérer des données ordonnées
- les arbres syntactiques, utilisés par les compilateurs pour l'évaluation du code source
- les arbres de décision, utilisés en intelligence artificielle
- les arbres de codage, utilises en théorie de l'information (i.e. le code Huffman)
- les arbres d'occupation spatiale, utilises en robotique pour la navigation ou en infographie pour l'affichage de scènes









Arbres et STL

Pas d'arbre dans la STL ?

- en fait, plusieurs conteneurs utilisent des arbres, par exemple `stl::set`

Arbres et STL

Pas d'arbre dans la STL ?

- en fait, plusieurs conteneurs utilisent des arbres, par exemple `stl::set`
- en revanche, la STL ne fournit pas une structure de type arbre avec une interface d'accès complète

Arbres et STL

Pas d'arbre dans la STL ?

- en fait, plusieurs conteneurs utilisent des arbres, par exemple `stl::set`
- en revanche, la STL ne fournit pas une structure de type arbre avec une interface d'accès complète
- une des raisons : il y a beaucoup de types d'arbres

Arbres et STL

Pas d'arbre dans la STL ?

- en fait, plusieurs conteneurs utilisent des arbres, par exemple `stl::set`
- en revanche, la STL ne fournit pas une structure de type arbre avec une interface d'accès complète
- une des raisons : il y a beaucoup de types d'arbres
- chaque type est intéressant pour certains contextes d'utilisation

Arbres et STL

Pas d'arbre dans la STL ?

- en fait, plusieurs conteneurs utilisent des arbres, par exemple `stl::set`
- en revanche, la STL ne fournit pas une structure de type arbre avec une interface d'accès complète
- une des raisons : il y a beaucoup de types d'arbres
- chaque type est intéressant pour certains contextes d'utilisation
- il est extrêmement important de comprendre les avantages et les inconvénients entre les différents variétés d'arbres

Binary search trees (BST)

Propriété de base

- Pour deux nœuds x et y, si $x \in left(y)$ alors $key(x) < key(y)$

Parcours de l'arbre

Binary search trees (BST)

Propriété de base

- Pour deux nœuds x et y, si $x \in \mathit{left}(y)$ alors $\mathit{key}(x) < \mathit{key}(y)$
- Pour deux nœuds x et y, si $x \in \mathit{right}(y)$ alors $\mathit{key}(x) > \mathit{key}(y)$

Parcours de l'arbre

Binary search trees (BST)

Propriété de base

- Pour deux nœuds x et y, si $x \in left(y)$ alors $key(x) < key(y)$
- Pour deux nœuds x et y, si $x \in right(y)$ alors $key(x) > key(y)$

Parcours de l'arbre

- opération très importante, qui visite dans un ordre précis tous les nœuds et qui leur applique éventuellement une opération

Binary search trees (BST)

Propriété de base

- Pour deux nœuds x et y, si $x \in left(y)$ alors $key(x) < key(y)$
- Pour deux nœuds x et y, si $x \in right(y)$ alors $key(x) > key(y)$

Parcours de l'arbre

- opération très importante, qui visite dans un ordre précis tous les nœuds et qui leur applique éventuellement une opération
- **parcours en inorder** : l'ordre est d'abord le sous-arbre gauche, puis le nœud courant, et par la suite le sous-arbre droit

Binary search trees (BST)

Propriété de base

- Pour deux nœuds x et y, si $x \in left(y)$ alors $key(x) < key(y)$
- Pour deux nœuds x et y, si $x \in right(y)$ alors $key(x) > key(y)$

Parcours de l'arbre

- opération très importante, qui visite dans un ordre précis tous les nœuds et qui leur applique éventuellement une opération
- **parcours en inorder** : l'ordre est d'abord le sous-arbre gauche, puis le nœud courant, et par la suite le sous-arbre droit
- **parcours en preorder** : l'ordre est d'abord le nœud courant, puis le sous-arbre gauche, et par la suite le sous-arbre droit

Binary search trees (BST)

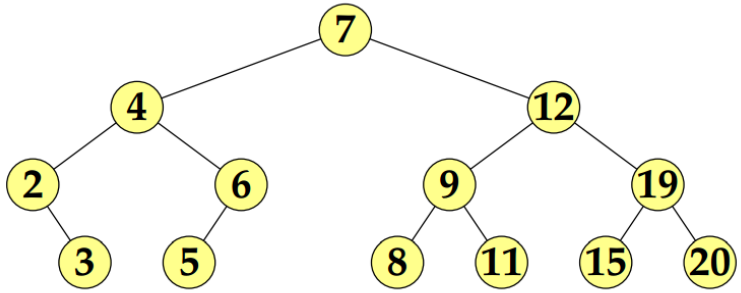
Propriété de base

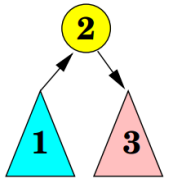
- Pour deux nœuds x et y, si $x \in left(y)$ alors $key(x) < key(y)$
- Pour deux nœuds x et y, si $x \in right(y)$ alors $key(x) > key(y)$

Parcours de l'arbre

- opération très importante, qui visite dans un ordre précis tous les nœuds et qui leur applique éventuellement une opération
- **parcours en inorder** : l'ordre est d'abord le sous-arbre gauche, puis le nœud courant, et par la suite le sous-arbre droit
- **parcours en preorder** : l'ordre est d'abord le nœud courant, puis le sous-arbre gauche, et par la suite le sous-arbre droit
- **parcours en postorder** : l'ordre est d'abord le sous-arbre gauche, puis le sous-arbre droit, et par la suite le nœud courant

Parcours en inorder



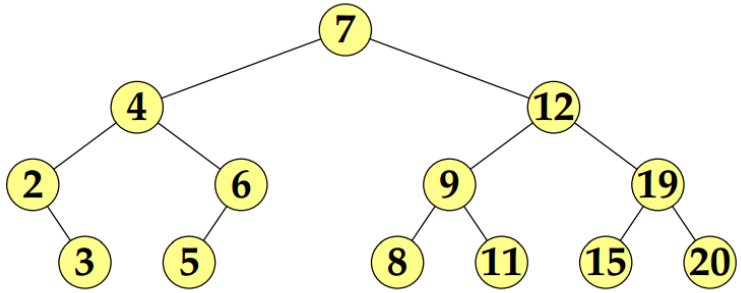


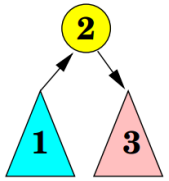
inorder

Quel est le parcours en inorder de l'arbre ?

```
function InOrder-Walk(a)
  if a = empty then return
  InOrder-Walk(left(a))
  key(a)
  InOrder-Walk(right(a))
```

Parcours en inorder



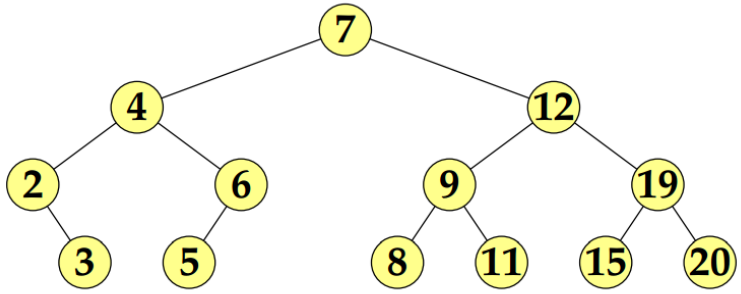


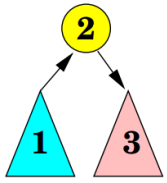
inorder

Quel est le parcours en inorder de l'arbre ? **2 3 4 5 6 7 8 9 11 12 15 19 20**

```
function InOrder-Walk(a)
  if a = empty then return
  InOrder-Walk(left(a))
  key(a)
  InOrder-Walk(right(a))
```

Parcours en inorder



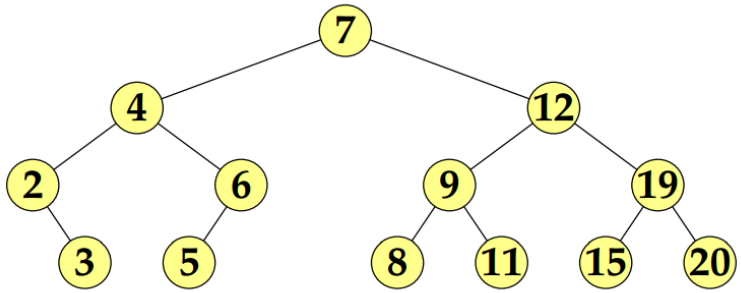


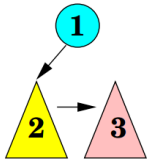
inorder

Quel est le parcours en inorder de l'arbre ? **2 3 4 5 6 7 8 9 11 12 15 19 20** Le parcours en inorder d'un BST trouve le clés en ordre croissant !

```
function InOrder-Walk(a)
  if a = empty then return
  InOrder-Walk(left(a))
  key(a)
  InOrder-Walk(right(a))
```

Parcours en preorder

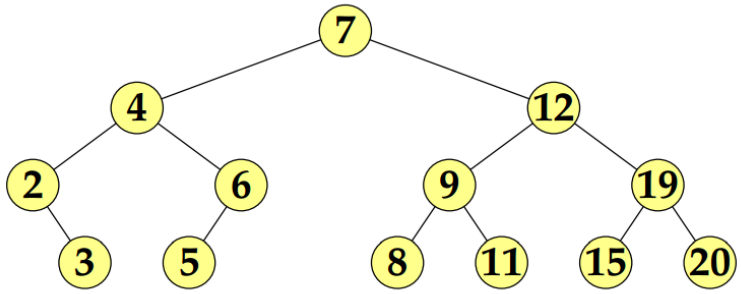


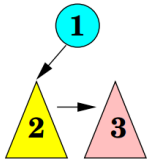


preorder

Quel est le parcours en preorder de l'arbre ?

Parcours en preorder

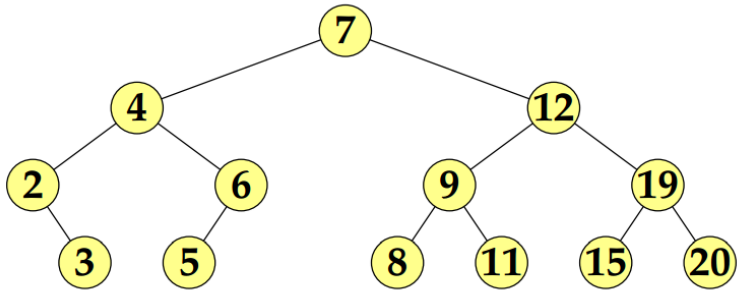


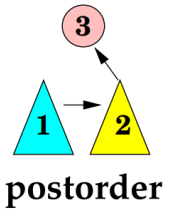


preorder

Quel est le parcours en preorder de l'arbre ? **7 4 2 3 6 5 12 9 8 11 19 15 20**

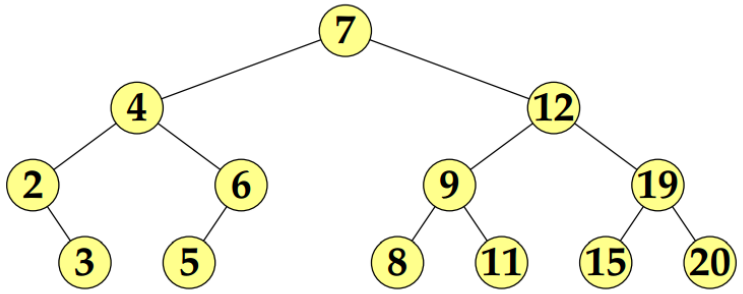
Parcours en postorder

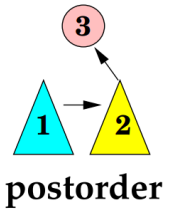




Quel est le parcours en postorder de l'arbre ?

Parcours en postorder

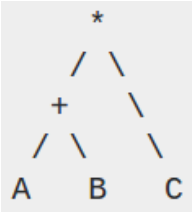




Quel est le parcours en postorder de l'arbre ? **3 2 5 6 4 8 11 9 15 20 19 12 7**

Une parenthèse : utilisation des parcours

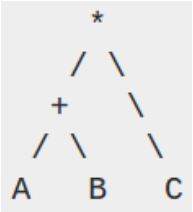
L'intérêt du parcours en inorder est clair pour un BST, mais a quoi servent les autres parcours ?
Les autres parcours sont très utiles, en fonction du contexte. Exemple :



- arbre syntactique construit à partir d'une expression arithmétique (A, B et C étant des valeurs numériques)

Une parenthèse : utilisation des parcours

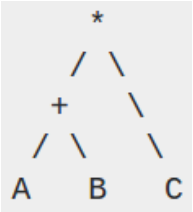
L'intérêt du parcours en inorder est clair pour un BST, mais a quoi servent les autres parcours ?
Les autres parcours sont très utiles, en fonction du contexte. Exemple :



- arbre syntactique construit à partir d'une expression arithmétique (A, B et C étant des valeurs numériques)
- **parcours en inorder** ?

Une parenthèse : utilisation des parcours

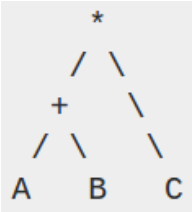
L'intérêt du parcours en inorder est clair pour un BST, mais a quoi servent les autres parcours ?
Les autres parcours sont très utiles, en fonction du contexte. Exemple :



- arbre syntactique construit à partir d'une expression arithmétique (A, B et C étant des valeurs numériques)
- **parcours en inorder** ?
- A+B*C

Une parenthèse : utilisation des parcours

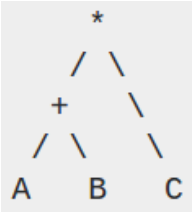
L'intérêt du parcours en inorder est clair pour un BST, mais a quoi servent les autres parcours ?
Les autres parcours sont très utiles, en fonction du contexte. Exemple :



- arbre syntactique construit à partir d'une expression arithmétique (A, B et C étant des valeurs numériques)
- **parcours en inorder** ?
- A+B*C
- **parcours en preorder** ?

Une parenthèse : utilisation des parcours

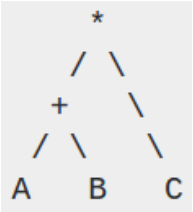
L'intérêt du parcours en inorder est clair pour un BST, mais a quoi servent les autres parcours ?
Les autres parcours sont très utiles, en fonction du contexte. Exemple :



- arbre syntactique construit à partir d'une expression arithmétique (A, B et C étant des valeurs numériques)
- **parcours en inorder** ?
- A+B*C
- **parcours en preorder** ?
- * + A B C

Une parenthèse : utilisation des parcours

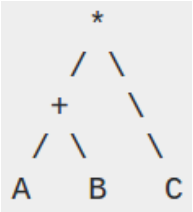
L'intérêt du parcours en inorder est clair pour un BST, mais a quoi servent les autres parcours ?
Les autres parcours sont très utiles, en fonction du contexte. Exemple :



- arbre syntactique construit à partir d'une expression arithmétique (A, B et C étant des valeurs numériques)
- **parcours en inorder** ?
- A+B*C
- **parcours en preorder** ?
- * + A B C
- **parcours en postorder** ?

Une parenthèse : utilisation des parcours

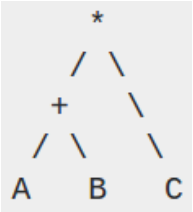
L'intérêt du parcours en inorder est clair pour un BST, mais a quoi servent les autres parcours ?
Les autres parcours sont très utiles, en fonction du contexte. Exemple :



- arbre syntactique construit à partir d'une expression arithmétique (A, B et C étant des valeurs numériques)
- **parcours en inorder** ?
- A+B*C
- **parcours en preorder** ?
- * + A B C
- **parcours en postorder** ?
- A B + C *

Une parenthèse : utilisation des parcours

L'intérêt du parcours en inorder est clair pour un BST, mais a quoi servent les autres parcours ?
Les autres parcours sont très utiles, en fonction du contexte. Exemple :



- arbre syntactique construit à partir d'une expression arithmétique (A, B et C étant des valeurs numériques)
- **parcours en inorder** ?
- A+B*C
- **parcours en preorder** ?
- * + A B C
- **parcours en postorder** ?
- A B + C *
- Quel est d'après vous le parcours qui permet le plus facilement une évaluation de l'expression, et comment ?

Complexités des opérations sur les BST

Devoir maison - pour chacune de ces opérations vous pouvez avoir des questions à l'examen. Montrer que la complexité de tous les opérations suivantes est $O(h)$, ou h est la profondeur de l'arbre BST, et écrire le code pour l'opération respective :

- recherche de la valeur minimale

Complexités des opérations sur les BST

Devoir maison - pour chacune de ces opérations vous pouvez avoir des questions à l'examen. Montrer que la complexité de tous les opérations suivantes est $O(h)$, ou h est la profondeur de l'arbre BST, et écrire le code pour l'opération respective :

- recherche de la valeur minimale
- recherche de la valeur maximale

Complexités des opérations sur les BST

Devoir maison - pour chacune de ces opérations vous pouvez avoir des questions à l'examen. Montrer que la complexité de tous les opérations suivantes est $O(h)$, ou h est la profondeur de l'arbre BST, et écrire le code pour l'opération respective :

- recherche de la valeur minimale
- recherche de la valeur maximale
- insertion d'une nouvelle valeur

Complexités des opérations sur les BST

Devoir maison - pour chacune de ces opérations vous pouvez avoir des questions à l'examen. Montrer que la complexité de tous les opérations suivantes est $O(h)$, ou h est la profondeur de l'arbre BST, et écrire le code pour l'opération respective :

- recherche de la valeur minimale
- recherche de la valeur maximale
- insertion d'une nouvelle valeur
- recherche du successeur d'une clé (la clé suivante en ordre croissant) à partir du nœud avec la clé initiale ; on considère qu'on peut remonter au parent d'un nœud si besoin

Complexités des opérations sur les BST

Devoir maison - pour chacune de ces opérations vous pouvez avoir des questions à l'examen. Montrer que la complexité de tous les opérations suivantes est $O(h)$, ou h est la profondeur de l'arbre BST, et écrire le code pour l'opération respective :

- recherche de la valeur minimale
- recherche de la valeur maximale
- insertion d'une nouvelle valeur
- recherche du successeur d'une clé (la clé suivante en ordre croissant) à partir du nœud avec la clé initiale ; on considère qu'on peut remonter au parent d'un nœud si besoin
- recherche du prédécesseur d'une clé (la clé suivante en ordre décroissant)

Complexités des opérations sur les BST

Devoir maison - pour chacune de ces opérations vous pouvez avoir des questions à l'examen. Montrer que la complexité de tous les opérations suivantes est $O(h)$, ou h est la profondeur de l'arbre BST, et écrire le code pour l'opération respective :

- recherche de la valeur minimale
- recherche de la valeur maximale
- insertion d'une nouvelle valeur
- recherche du successeur d'une clé (la clé suivante en ordre croissant) à partir du nœud avec la clé initiale ; on considère qu'on peut remonter au parent d'un nœud si besoin
- recherche du prédécesseur d'une clé (la clé suivante en ordre décroissant)
- suppression d'un nœud

Complexités des opérations sur les BST

Devoir maison - pour chacune de ces opérations vous pouvez avoir des questions à l'examen. Montrer que la complexité de tous les opérations suivantes est $O(h)$, ou h est la profondeur de l'arbre BST, et écrire le code pour l'opération respective :

- recherche de la valeur minimale
- recherche de la valeur maximale
- insertion d'une nouvelle valeur
- recherche du successeur d'une clé (la clé suivante en ordre croissant) à partir du nœud avec la clé initiale ; on considère qu'on peut remonter au parent d'un nœud si besoin
- recherche du prédécesseur d'une clé (la clé suivante en ordre décroissant)
- suppression d'un nœud

Comparer la complexité de ces opérations avec celle des opérations équivalentes sur les conteneurs de base déjà rencontres : vecteurs, listes

Complexités des opérations sur les BST

Devoir maison - pour chacune de ces opérations vous pouvez avoir des questions à l'examen. Montrer que la complexité de tous les opérations suivantes est $O(h)$, ou h est la profondeur de l'arbre BST, et écrire le code pour l'opération respective :

- recherche de la valeur minimale
- recherche de la valeur maximale
- insertion d'une nouvelle valeur
- recherche du successeur d'une clé (la clé suivante en ordre croissant) à partir du nœud avec la clé initiale ; on considère qu'on peut remonter au parent d'un nœud si besoin
- recherche du prédécesseur d'une clé (la clé suivante en ordre décroissant)
- suppression d'un nœud

Comparer la complexité de ces opérations avec celle des opérations équivalentes sur les conteneurs de base déjà rencontres : vecteurs, listes
Théorème essentiel : la profondeur moyenne d'un arbre BST construit à partir de n éléments aléatoires est $O(\log n)$